```
temperature_input = 28

temp_low_level = fuzz.interp_membership

temp_medium_level = (temp_medium, temperature

temp_high_level = fuzz.interp_membership
                  (temperature, temp_high, temp_

fan_low_level = temp_low_level

fan_medium_level = temp_medium_level

fan_high_level = temp_high_level

fan_output = fuzz.defuzz(fan_speed,
             fan_low * fan_low_level + fan_med
             * fan_medium_level + fan_high *
             'centroid')

print(f"Temperature: {temperature_input}°C")

print(f"Fuzzified fan speed: {fan_output
```

Result :-

The output is

Temperature : 28°C

Fuzzified fan speed = 6.00

Exp no:-11 Implementation of Fuzzy Inference System

Aim:

To Implement Fuzzy Inference System

Program:

```
import numpy as np
import skfuzzy as fuzz
import matplotlib.pyplot as plt
temperature = np.arange(0,41,1)
fan_speed = np.arange(0,11,1)
temp_low = fuzz.trimf(temperature,[0,0,20])
temp_medium = fuzz.trimf(temperature,[10,20,30])
temp_high = fuzz.trimf(temperature,[20,30,40])
plt.figure(figsize=(10,6))
plt.subplot(2,1,1)
plt.plot(temperature, temp_low, label='Low')
plt.plot(temperature, temp_high, label='High')
plt.title('Temperature Functions')
plt.xlabel('Temperature (°C)')
plt.tight_layout()
plt.show()
```

Result :-

Output :

The goal 'f' can be achieved

The goal 'e' cannot be achieved

The goal 'd' can be achieved

Block words Program

Aim :

To implement the block words Program

Program :-

```
class Blocksworld
    def __init__(self, num_blocks):
        self.state = [[block] for block in range
                      (num_blocks)]
        self.num_block = num_blocks
    def display_state(self):
        for state in self.state:
            print(f"Block(s) on Stack: {stack}")
    def move(self, block, destination):
        destination_stack = self.find_block
    def find_block(self, block):
        for stack in self.stack:
            if block in stack:
                return stack
        return None
    def goal_state(self, goal):
        self.state = goal
```

Output

Initial State :

Block (s) on Stack : [0]

Block (s) on Stack : [1]

Block (s) on Stack : [2]

Goal State set

Block (s) on Stack : [0, 1]

Block (s) on Stack : [2]

Performing Moves :

Block (s) on Stack : [1]

Block (s) on Stack : [2, 0]

Block (s) on Stack : []

Block (s) on Stack : [1]

Block (s) on Stack : [2, 0]

Block (s) on Stack : []

Block (s) on Stack : [1, 2]

Block [s] on Stack : [0]

Block [s] on Stack : []

Output :

Expected output for the goal 'f':

The goal 'f' cannot be achieved

Expected output for goal 'd':

The goal 'd' can be achieved

Aim :-

To implement forward chaining algorithm.

Program :-

```
facts = {'a': True, 'b': True, 'c': False}

rules : [('d', ['a', 'b']), ('e', ['b', 'c']),
         ('f', ['d', 'e'])]

def forward_chaining(facts, rules, goal):
    for rule in rules:
        head, body = rule
        if head not in inferred and all
        (fact in inferred for fact in body):
        current_inferred.add(head)
        if current_inferred:
    else
        new_inferred = set()
    return goal in inferred

goal = 'f'
if forward_chaining(facts, rule, goal):
    print(f"The goal {goal} can be
```

Result :

output :

Knowledge Base : [["~p", "Q"],
["p"], ["~Q", "R"], ["~R"]
]

Query : ["R"]

Knowledge Base : [["P", "Q"], ["p"], ["~q", "R"],
["~R"]]

Query : ["R"]

Aim :-

To implement backward chaining

Program :-

```
facts = {'a': True, 'b': True, 'c': False}

rules = [('d', ['a', 'b']), ('e', ['b', 'c']),
         ('f', ['d', 'c'])]

def backward_chaining(goal, facts, rules):
    if goal in facts:
        return facts[goal]

    for rule in rules:
        head, body = rule
        if head == goal
            if all(backward_chaining(fact, fa
                                     rul
                   for fact in body):
                return True
    return False

if backward_chaining(goal, facts, rule),
    print(f"The goal "{goal}' ca be ach

else:
    print(f"The goal '{goal}' cannot.")
```

Expno:-07 Implementation of Unification and Resolution algorithm

Aim:-

To implement unification and resolution algorithm.

Program

```
def unify(var1, var2, subst):
    if subst is None:
        return None
    elif var1 == var2:
        return subst
    elif isinstance(var1, str) and var1.islower():
        return unify_var(var2, var1, subst)
    elif isinstance(var1, list) and isinstance
    (var2, list) and len(var1) == len(var2):
        return unify(var1[1:], var2[1:],
            unify(var1[0], var2[0], subst))
    else:
        return None
def occur_check(var, x, subst):
    if var == x:
        return True
```

```
elif isinstance(x, list):
    return any(occurs_check(var, xi, subst) for xi in x)
return False

def negate(literal):
    if isinstance(literal, list):
        return [negate(lit) for lit in literal]
    else:
        return f"~{literal}"

if __name__ == "__main__":
    knowledge_base = [
        ["~p", "q"],
        ["p"],
        ["~q", "r"],
        ["~r"]
    ]
    print("Knowledge Base:", knowledge_base)
    print("Query:", query)
    if result:
        print("The query is satisfiable.")
    else:
        print("The query is not satisfiable.")
```

Exp no: 06
Date 9/9/26

Implementation of A* Algorithm

Aim :-

To implement the working of A* Algorithm

Program

```
import heapq
class Node
    def __init__(self, state, parent, g, h):
        self.state = state
        self.parent = parent
        self.g = g
        self.h = h
        self.f = g+h
    def __lt__(self, other):
        return self.f < other.f
def a_star_search(start, goal, heuristic, neighbors):
    open_list = []
    closed_list = set()
    start_node = Node(start, None, 0, heuristic
                      (start, goal))
    heapq.heappush(open_list, start_node)
while open_list:
    current_node = heapq.heappop(open_list)
```

```
        return path[::-1]
    closed_list.add(current_node.state)

for neighbour, cost in neighbours(current_node.state):
    if neighbour in closed_list:
        continue
    g = current_node.g + cost
    h = heuristic(neighbour, goal)

return None

def heuristic(state, goal):
    return abs(state[0]-goal[0])
           +abs(state[1]-goal[1])

def neighbours(state):
    x, y = state
    start = (0,0)
    goal = (3,3)
    path("Path from start to goal :", path)
```

output

for start = (0,0) and goal = (3,3)

Path from start to goal: [(0,0),

(1,0), (2,0), (3,0), (3,1), (3,2), (3,3)]

```
        return best

    else:
        best = float('inf')
        for i in range(2):
            value = minimax(
                depth + 1,
                node_index * 2 + i,
                True,
                value,
                alpha,
                beta
            )
            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
                break
        return best

values = [3, 5, 6, 9, 1, 2, 0, -1]

alpha = float('-inf')
beta = float('inf')

result = minimax(0, 0, True, values, alpha, beta)
```

```
print ("optimal value:", result)
```

Output:

optimal value: 5

Result:

Thus, the program was implemented successfully and the optimal value is 5.

```
values = [3, 5, 2, 9, 12, 5, 23, 23]

height = 3

result = minMax(0, 0, True, values, height)

Print("optimal value :", result)
```

OUTPUT :

optimal value : 12

Result :

Thus the minmax algorithm was successfully implemented and the optimal value was obtained as 12.

Exp: 04 Implementation of Alpha Beta Pruning

Aim:-

To implement alpha beta pruning optimization technique for minimax Algorithm

Program:-

```
def minimax (depth, node_index, maximizing_player, values, alpha, beta
    if depth == 3:
        return values[node_index]

    if maximizing_player:
        best = float('-inf')

        for i in range(2):
            value = minimax(
                depth+1,
                node_index*2+i,
                False,
                values,
                alpha,
                beta
            )
        best = max(best, value)
        alpha = max(alpha, best)

        if beta <= alpha:
```

Expno: 03 Minmax algorithm Implementation

Aim:

To implement the min-max algorithm for finding the optimal move in a two-player game

Program:

```
def minmax(depth, node_index, maximizing_player,
           values, height):

    if depth == height:
        return values[node_index]

    if maximizing_player:
        return max(
            minmax(depth+1, node_index*2, False,
                   values, height)
        )
    else:
        return min(
            minmax(depth+1, node_index*2,
                   True, values, height),
            minmax(depth+1, node_index*2+1,
                   values, height)
        )
```

```
graph: {
    0 : [1, 2];
    1 : [0, 3, 4],
    2 : [0],
    3 : [1],
    4 : [1]
}
bfs (graph, 0)
```

Program:

Dfs

```
def dfs(graph, node, visited):
    visited.add(node)
    print(node) end: " "
    for neighbour not in visited:
        dfs (graph, neighbour, visited)
```

```
graph = {
    0 : [1, 2],
    10: [0, 3, 4],
    20: [0],
    30: [1],
    4 : [1]
}
```

visited = set()

dfs(graph, 0, visited)

output:

0 1 3 4 2

Result :-

Thus, the BFS and DFS using queue and recursion are implemented successfully

29/7/26

Exp: 2

Breadth first Search and Depth first Search

Aim:-

To implement BFS and DFS using queing and recursion in python.

Program;

BFS:-

```
from collections import deque

def bfsc graph, start ):

    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end " ")
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
```

Result :-

Thus, the 8- Queen problem has been solved using backtracking algorithm.